

Introduction to Algorithms

Email: grad.saeed@gmail.com

Github: github.com/saeedgrad

Solving recurrences

3- Master theorem:

The *master method* (Sections 4.5 and 4.6) is the easiest method, when it applies. It provides bounds for recurrences of the form

$$T(n) = aT(n/b) + f(n) ,$$

where $a > 0$ and $b > 1$ are constants and $f(n)$ is a given “driving” function. This type of recurrence tends to arise more frequently in the study of algorithms than any other. It characterizes a divide-and-conquer algorithm that creates a subproblems, each of which is $1/b$ times the size of the original problem, using $f(n)$ time for the divide and combine steps. To apply the master method, you need to memorize three cases, but once you do, you can easily determine asymptotic bounds on running times for many divide-and-conquer algorithms.

Merge sort

1. **Divide:** Trivial.
2. **Conquer:** Recursively sort 2 subarrays.
3. **Combine:** Linear-time merge.

$$T(n) = 2T(n/2) + \Theta(n)$$

subproblems

subproblem size

work dividing and combining

github.com/saeedgrad

Master theorem

$$T(n) = a T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \varepsilon})$, constant $\varepsilon > 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a}) .$$

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, constant $k \geq 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n) .$$

CASE 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$, constant $\varepsilon > 0$,

and regularity condition

$$\Rightarrow T(n) = \Theta(f(n)) .$$

Master theorem

$$T(n) = a T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \varepsilon})$, constant $\varepsilon > 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a}) .$$

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, constant $k \geq 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n) .$$

CASE 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$, constant $\varepsilon > 0$,

and regularity condition

$$\Rightarrow T(n) = \Theta(f(n)) .$$

Merge sort: $a = 2, b = 2 \Rightarrow n^{\log_b a} = n^{\log_2 2} = n$

$$\Rightarrow \text{CASE 2 } (k = 0) \Rightarrow T(n) = \Theta(n \lg n) .$$

Binary search

Find an element in a sorted array:

- 1. Divide:* Check middle element.
- 2. Conquer:* Recursively search 1 subarray.
- 3. Combine:* Trivial.

Binary search

Find an element in a sorted array:

- 1. Divide:* Check middle element.
- 2. Conquer:* Recursively search 1 subarray.
- 3. Combine:* Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

- 1. Divide:* Check middle element.
- 2. Conquer:* Recursively search 1 subarray.
- 3. Combine:* Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search 1 subarray.
- 3. Combine:** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search 1 subarray.
- 3. Combine:** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

- 1. Divide:* Check middle element.
- 2. Conquer:* Recursively search 1 subarray.
- 3. Combine:* Trivial.

Example: Find 9

3 5 7 8 9 12 15

Binary search

Find an element in a sorted array:

- 1. Divide:* Check middle element.
- 2. Conquer:* Recursively search 1 subarray.
- 3. Combine:* Trivial.

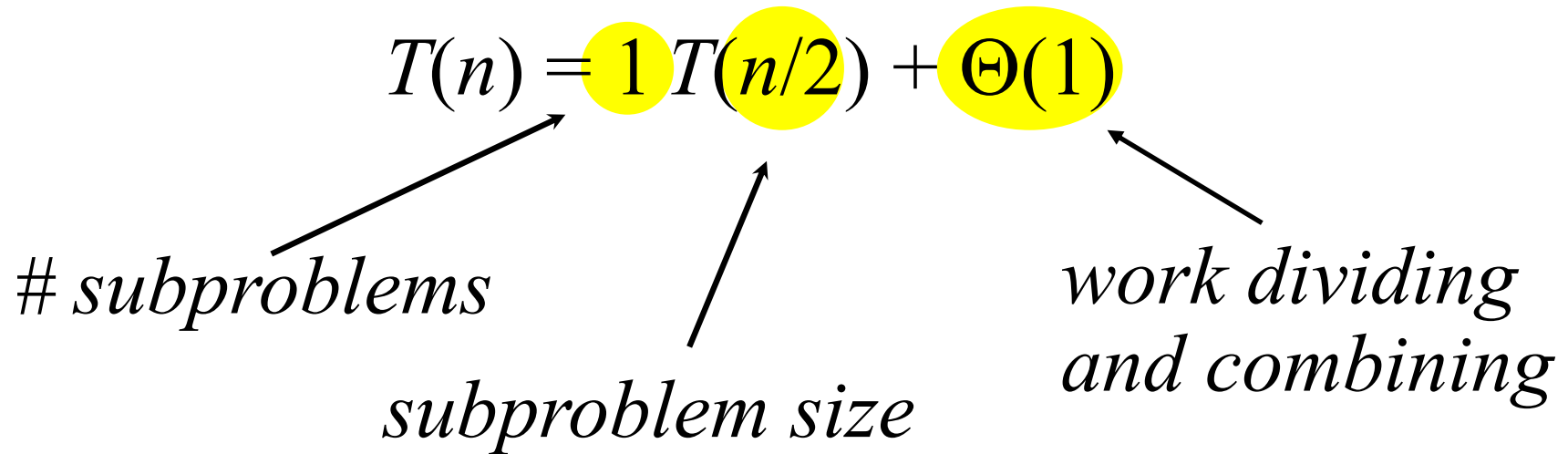
Example: Find 9

3 5 7 8  12 15

Recurrence for binary search

$$T(n) = 1 T(n/2) + \Theta(1)$$

subproblems *subproblem size* *work dividing and combining*



Recurrence for binary search

$$T(n) = 1 T(n/2) + \Theta(1)$$

subproblems *subproblem size* *work dividing and combining*

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow \text{CASE 2 } (k = 0)$$
$$\Rightarrow T(n) = \Theta(\lg n) .$$

Powering a number

Problem: Compute a^n , where $n \in \mathbf{N}$.

Naive algorithm: $\Theta(n)$.

github.com/saeedgrad

Powering a number

Problem: Compute a^n , where $n \in \mathbf{N}$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \times a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & \text{if } n \text{ is odd.} \end{cases}$$

Powering a number

Problem: Compute a^n , where $n \in \mathbf{N}$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \times a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & \text{if } n \text{ is odd.} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\lg n) .$$

Master theorem (Straightforward Version)

$$T(n) = a T(n/b) + f(n)$$

for positive integers $a \geq 1$ and $b > 1$ and where $\frac{n}{b}$ can mean either $\lfloor \frac{n}{b} \rfloor$ or $\lceil \frac{n}{b} \rceil$ it doesn't matter which. Then we have:

CASE 1:

If $f(n) = \mathcal{O}(n^c)$ and $\log_b a > c$ then $T(n) = \Theta(n^{\log_b a})$.

CASE 2:

If $f(n) = \Theta(n^c)$ and $\log_b a = c$ then $T(n) = \Theta(n^{\log_b a} \lg n)$.

If $f(n) = \Theta(n^c \lg^k n)$ and $\log_b a = c$ then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

CASE 3:

If $f(n) = \Omega(n^c)$ and $\log_b a < c$ then $T(n) = \Theta(f(n))$.

Example 1

$$T(n) = 4T(n/2) + n^2 + \lg n$$

github.com/saeedgrad

Example 1

$$T(n) = 4T(n/2) + n^2 + \lg n$$

Observe that $f(n) = n^2 + \lg n$ and $\log_b a = \log_2 4 = 2$ and then:

- $f(n) = \Theta(n^2)$ with $c = 2$:

Observe $\log_2 4 = 2 = c$ so Case 2 applies and so:

$$T(n) = \Theta(n^{\log_2 4} \lg n) = \Theta(n^2 \lg n)$$

git

Example 2

$$T(n) = 2T(n/8) + n$$

github.com/saeedgrad

Example 2

$$T(n) = 2T(n/8) + n$$

Observe that $f(n) = n$ and $\log_b a = \log_8 2 = 1/3$ and then:

- $f(n) = \Theta(n)$ with $c = 1$:
Observe $\log_8 2 = 1/3 \neq 1 = c$ so Case 2/2f do not apply.
- $f(n) = \mathcal{O}(n)$ with $c = 1$:
Observe $\log_8 2 = 1/3 \not\geq 1 = c$ so Case 1 does not apply.
- $f(n) = \Omega(n)$ with $c = 1$:
Observe $\log_8 2 = 1/3 < 1 = c$ so Case 3 applies and so:

$$T(n) = \Theta(f(n)) = \Theta(n)$$

Example 3

$$T(n) = 3T(n/3) + \sqrt{n} + 1$$

github.com/saeedgrad

Example 3

$$T(n) = 3T(n/3) + \sqrt{n} + 1$$

Observe that $f(n) = \sqrt{n} + 1$ and $\log_b a = \log_3 3 = 1$ and then:

- $f(n) = \Theta(n^{1/2})$ with $c = 1/2$:
Observe $\log_3 3 = 1 \neq 1/2 = c$ so Case 2/2f do not apply.
- $f(n) = \mathcal{O}(n^{1/2})$ with $c = 1/2$:
Observe $\log_3 3 = 1 > 1/2 = c$ so Case 1 applies and so:

$$T(n) = \Theta(n^{\log_3 3}) = \Theta(n)$$

Example 4

$$T(n) = 3T(n/4) + n \lg n + n$$

github.com/saeedgrad

Example 4

$$T(n) = 3T(n/4) + n \lg n + n$$

Observe that $f(n) = n \lg n + n$ and $\log_b a = \log_4 3 \approx 0.*$ and then:

- $f(n) = \Theta(n \lg n)$ with $c = 1$:
Observe $\log_4 3 \approx 0.* \neq 1 = c$ so Case 2/2f do not apply.
- $f(n) = \mathcal{O}(n^2)$ with $c = 2$:
Observe $\log_4 3 \approx 0.* \not\geq 2 = c$ so Case 1 does not apply.
- $f(n) = \Omega(n)$ with $c = 1$:
Observe $\log_4 3 \approx 0.* < 1 = c$ so Case 3 applies and so:

$$T(n) = \Theta(f(n)) = \Theta(n \lg n)$$

Example 5

$$T(n) = 10T(n/2) + n^2 \lg n + n^2 + 1$$

github.com/saeedgrad

Example 5

$$T(n) = 10T(n/2) + n^2 \lg n + n^2 + 1$$

Observe that $f(n) = n^2 \lg n + n^2 + 1$ and $\log_b a = \log_2 10 \approx 3.*$

- $f(n) = \Theta(n^2 \lg n)$ with $c = 2$:

Observe $\log_2 10 \approx 3.* \neq 2 = c$ so Case 2/2f do not apply.

- $f(n) = \mathcal{O}(n^3)$ with $c = 3$:

Observe $\log_2 10 \approx 3.* > 3 = c$ so Case 1 applies and so:

$$T(n) = \Theta(n^{\log_2 10})$$

Example 6

$$T(n) = 9T(n/3) + n^2 \lg n + \lg n$$

Example 6

$$T(n) = 9T(n/3) + n^2 \lg n + \lg n$$

Observe that $f(n) = n^2 \lg n + \lg n$ and $\log_b a = \log_3 9 = 2 \varepsilon$

- $f(n) = \Theta(n^2 \lg n)$ with $c = 2$:

Observe $\log_3 9 = 2 = c$ so Case 2f applies with $k = 1$

$$T(n) = \Theta(n^{\log_3 9} \lg^2 n) = \Theta(n^2 \lg^2 n)$$